

UPSA: A Human-Centric Unified Privacy & Security Assistant with Adaptive Machine Learning

Muhammad Bilal Badar¹, Abu Huraira², and Huzaifa Asad³

Department of Cyber Security, BSCYS-F-23-B, Islamabad, Pakistan

Abstract

Traditional security platforms impose rigid, one-size-fits-all controls that consistently frustrate users, producing workarounds and policy non-compliance. This paper presents the Unified Privacy & Security Assistant (UPSA), a production-ready web application that operationalizes usable security principles through an adaptive, human-centric architecture. Built on Flask 3.x with Python 3.12+, UPSA integrates ten fully functional modules: adaptive risk-based authentication, privacy health scoring, user frustration analysis, browser permission auditing, natural-language privacy policy analysis, behavioural analytics, an ensemble machine learning classifier, a security awareness survey, and System Usability Scale evaluation. The platform's Adaptive Risk Engine evaluates six behavioural dimensions at login—new device, new browser, new location, unusual time, failed attempts, and impossible travel—mapping aggregate scores to three authentication outcomes: direct login (score ≤ 30), MFA challenge (31–60), and blocked access (> 60). A unified Privacy Health Score aggregating eight independent security factors provides non-technical users with immediate, actionable insight into their security posture. The Frustration Analyzer actively monitors UX friction signals and programmatically relaxes non-essential security prompts when frustration thresholds are exceeded, addressing a documented root cause of security policy circumvention. An integrated ML ensemble (Random Forest, Logistic Regression, Decision Tree) predicts phishing susceptibility and account takeover risk with personalized plain-language explanations. Experimental evaluation demonstrates that UPSA achieves a System Usability Scale score in the 'Good' to 'Excellent' range, validating the core thesis that adaptive, explainable security systems can achieve superior outcomes on both usability and protection dimensions simultaneously.

Index Terms: Adaptive Authentication, Human-Computer Interaction, Machine Learning, Privacy Health Score, Usable Security, Web Application Security.

I. INTRODUCTION

The digital landscape presents users with an escalating sequence of security demands: multi-factor authentication prompts, browser permission dialogs, opaque privacy policies in dense legal language, and vigilance against increasingly sophisticated phishing attacks. Despite decades of research in usable security, a persistent tension between usability and security impairs both user experience and organizational security outcomes [1].

Studies in human-computer interaction (HCI) and usable security consistently demonstrate that overly burdensome security controls drive coping strategies that undermine the very protections those controls are intended to provide [6]. Users confronted with repeated MFA challenges abandon sessions, share credentials, or disable security features. Privacy policies spanning dozens of pages in legal terminology are rarely read, leaving users uninformed of significant data practices [11].

UPSA addresses this documented failure mode through a central thesis: security systems can be simultaneously stronger and more usable if they adapt to context, explain themselves in plain language, and treat user frustration as a first-class signal rather than an irrelevant side effect. The platform observes, measures, explains, predicts, and adapts — transitioning from a reactive, enforcement-only model to a proactive, human-centric partnership model.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the research gaps addressed by this study. Section IV describes the system architecture and methodology. Section V presents experimental results and module analysis. Section VI discusses industry applications. Section VII addresses limitations and future work. Section VIII concludes.

II. RELATED WORK

Whitten and Tygar [1] established the foundational challenge: security tools designed for expert users consistently fail in the hands of general users, as demonstrated in their landmark evaluation of PGP 5.0. Their observation that "Johnny can't encrypt" remains relevant to modern security interfaces.

Cranor and Garfinkel [2] systematized the field of security usability, establishing design principles for systems that must balance protection with accessibility. Their framework continues to inform the architecture of human-centric security systems. Adams and Sasse [6] further demonstrated that users are not inherently careless — rather, they make rational decisions within the constraints they perceive, often circumventing security controls when those controls are poorly designed.

Bonneau et al. [5] conducted a comprehensive comparative evaluation of web authentication schemes, highlighting the persistent trade-offs between security, usability, and deploy-ability. Their work establishes the theoretical foundation for adaptive authentication approaches that dynamically calibrate challenge levels based on contextual risk.

McDonald and Cranor [11] quantified the cost of reading privacy policies, finding that full comprehension would require impractical time investments from the average user. This finding directly motivates machine-assisted policy summarization.

Schaub et al. [8] explored the design space for effective privacy notices, identifying contextual integrity and brevity as key factors in user comprehension.

Palun and Tambi [10] examined adaptive multi-factor authentication in enterprise environments, demonstrating that risk-scored authentication can reduce friction for low-risk logins while maintaining protection against anomalous access. Their risk factor taxonomy directly informs UPSA's risk engine design. The OWASP Top Ten [14] and NIST SP 800-63B [15] provide the security standards baseline against which UPSA's authentication implementation is measured.

III. RESEARCH GAPS

The key research gaps addressed by this study are summarized in Table I.

Table I: Key Research Gaps Addressed by this Study

S.No.	Gap Category	References	This Study's Contribution
1	Static Authentication	[1], [5], [14]	Adaptive Risk Engine with six behavioural dimensions routing users through stepped-up authentication flows
2	Security Opacity	[6], [9]	Plain-language explanations and unified Privacy Health Score for non-technical users
3	Policy Inaccessibility	[8], [11]	Rule-based NLP Policy Analyzer with structured risk summaries across five dimensions
4	Reactive Security	[2], [10]	ML Classifier ensemble predicting susceptibility before exploitation with personalized explanations

IV. MATERIAL AND METHODS

A. System Architecture

UPSA is designed as a layered, modular Flask application following the Application Factory pattern. This architectural choice enables environment-specific configuration injection, simplified unit testing with in-memory databases, and clean separation of concerns across five architectural layers: Frontend, Application, Service, Data, and Security.

1) Frontend Layer

The presentation tier is built with Bootstrap 5.3, vanilla JavaScript, and Chart.js. All interactive data visualizations use Chart.js with AJAX endpoints defined in the Analytics route, eliminating page reloads for time-range switching.

2) Application Layer

Flask 3.x routes are organized into twelve Blueprints (auth, dashboard, privacy, risk, analytics, recommendations, security, survey, policy, settings, admin, API), each registered with its own URL prefix, enabling fine-grained rate limiting and permission decorators.

3) Service Layer

The service layer encapsulates all business logic, keeping routes thin. Key services include: auth_service, risk_engine, privacy_service, ml_service, frustration_service, policy_analyzer, and audit_service. All services interact with the database exclusively through SQLAlchemy models.

4) Data Layer

SQLAlchemy ORM with Flask-Migrate manages the persistence tier across fifteen interrelated tables. SQLite is the default engine, architected for seamless migration to PostgreSQL via a single DATABASE_URL environment variable change.

5) Security Layer

Cross-cutting security is handled at the Flask extension level: Flask-WTF provides CSRF validation; Flask-Limiter enforces IP-based rate limits; Werkzeug bcrypt handles password hashing at cost factor 12; Flask-Login manages session lifecycle.

B. Configuration Management

Three configuration classes (DevelopmentConfig, TestingConfig, ProductionConfig) inherit from a shared base Config. Table II presents key security-relevant configuration parameters.

Table II: Key Security Configuration Parameters

Configuration Parameter	Value	Notes
SECRET_KEY	Environment variable	Required in production
BCRYPT_LOG_ROUNDS	12 (dev), 13 (prod)	Higher = more secure
SESSION_COOKIE_SECURE	False (dev), True (prod)	HTTPS enforcement
MAX_LOGIN_ATTEMPTS	5	Before account lockout

LOCKOUT_DURATION	900 seconds (15 min)	Auto-reset on timeout
MFA_CODE_EXPIRY	300 seconds (5 min)	Single-use codes
RATELIMIT_DEFAULT	200 per hour	IP-based enforcement
LOGIN_RATE_LIMIT	5 per 5 minutes	Brute force protection
MAX_CONTENT_LENGTH	10 MB	File upload cap
ALLOWED_EXTENSIONS	pdf, docx, txt	Policy document types

C. Adaptive Risk Engine

The risk engine (app/services/risk_engine.py) evaluates six independent risk factors at every login attempt. The score accumulation and threshold mapping follows Equation (1):

$$\text{Risk Score} = \sum (\text{Factor}_i \times \text{Weight}_i), \text{ where } i \in \{1, \dots, 6\}$$

The six factors and their maximum point values are: New Device (25 pts), New Browser (20 pts), New Location (30 pts), Unusual Time (15 pts), Failed Attempts (25 pts), and Impossible Travel (40 pts). The scoring thresholds map directly to authentication outcomes as per the decision function:

$$\text{Auth_Outcome} = \{\text{Allow} \mid \text{score} \leq 30, \text{MFA} \mid 31 \leq \text{score} \leq 60, \text{Block} \mid \text{score} > 60\}$$

D. Privacy Health Score

The Privacy Health Analyzer computes a composite score $H \in [0, 100]$ by aggregating eight independent security dimensions. The score is formally defined as:

$$H = \sum (\text{Component}_j), \text{ where } j \in \{\text{MFA}, \text{Password Strength}, \text{Password Age}, \text{Permissions}, \text{Survey}\}$$

Score snapshots are persisted at every recommendation completion event, enabling trend analysis over the 180-day historical window seeded for each user.

E. Machine Learning Classifier

The ML subsystem implements an ensemble combining Random Forest (100 estimators), Logistic Regression (max_iter=1000), and Decision Tree models. The 14-dimensional feature vector x_i is defined as:

$$x_i = [Q1, \dots, Q10, \text{MFA_enabled}, \text{password_age_norm}, \text{login_failures_norm}, \text{privacy_score_norm}]$$

Phishing susceptibility $\hat{P}$ is predicted using averaged class probabilities from RF and LR models: $\hat{P} = (\hat{P}_{\text{RF}} + \hat{P}_{\text{LR}}) / 2$. Account takeover risk is predicted as a 0–1 probability by a dedicated Random Forest model. All models are trained on a 500-sample synthetic dataset and persisted to disk with joblib for immediate availability.

F. System Usability Scale

The SUS module implements the standardized evaluation instrument. Ten Likert-scale responses are scored using the standard formula:

$$\text{SUS} = 2.5 \times \sum [(r_{\text{odd}} - 1) + (5 - r_{\text{even}})]$$

Scores ≥ 81 are rated 'Excellent', 69–80 as 'Good', 51–68 as 'Average', and < 51 as 'Poor', benchmarked against the industry average of 68 [3, 4].

V. RESULTS AND DISCUSSION

A. Core Module Analysis

UPSA's ten modules collectively address the full lifecycle of user-centric security management. To ensure accurate, effective, and reliable operation in practical settings, each module was independently tested and validated against a defined test suite. The development team verified correct function in both isolation and integrated system contexts through rigorous testing.

B. Database Schema

The relational schema consists of fifteen tables designed on three principles: each table represents a single coherent domain concept; all foreign keys enforce referential integrity with cascade delete; and indexes are applied to high-frequency query columns. Table III summarizes the schema.

Table III: Database Table Summary

Table	Primary Purpose	Key Relationships
users	Core account & auth state	One-to-many with all other tables
privacy_scores	Health score history	Belongs to users
risk_events	Login risk assessments	Belongs to users

trusted_devices	Device trust registry	Belongs to users
frustration_metrics	UX friction tracking	Belongs to users
survey_responses	Security awareness answers	Belongs to users
sus_responses	SUS evaluation results	Belongs to users
ml_predictions	ML risk predictions	Belongs to users
recommendations	Actionable security tips	Belongs to users
policy_reports	Analysed policy documents	Belongs to users
audit_logs	Action audit trail	Belongs to users
notifications	In-app alert messages	Belongs to users
website_permissions	Browser permission audits	Belongs to users
email_verifications	Email token store	Belongs to users
password_resets	Reset token store	Belongs to users

C. Risk Engine Evaluation

The risk engine was validated across key login scenarios. Testing confirmed correct factor attribution across all six behavioural dimensions and accurate threshold boundary evaluation. Table IV presents the authentication outcome mapping.

Table IV: Risk Score to Authentication Outcome Mapping

Score Range	Risk Category	Auth Outcome	User Action
0 – 30	Low	Allowed	Direct login, session created
31 – 60	Medium	MFA Required	6-digit code sent via email
61 – 155	High	Blocked	Login denied, support redirect

D. Privacy Health Score Components

The eight-component Privacy Health Score was evaluated against component weightings designed to reflect security impact relative to total user risk exposure. Table V details each component's maximum contribution.

Table V: Privacy Health Score Component Breakdown

Score Component	Maximum Points & Calculation Logic
MFA Enabled	20 pts — Full if MFA active, 0 if not
Password Strength	25 pts — Stored password_strength field (0–25)
Password Age	15 pts — 15 if <90 days, 8 if <180 days, 0 if older
Browser Permissions	15 pts — 15 minus 3 per contextually inappropriate permission
Location Permissions	10 pts — 10 minus 3 per inappropriate location grant
Camera / Microphone	5 pts — 5 minus 2 per inappropriate cam/mic grant
Notification Permissions	5 pts — 5 minus 2 per inappropriate notification grant
Survey Awareness	15 pts — Normalized from security survey awareness score

E. Machine Learning Models

The ML classifier ensemble was trained on synthetic behavioural data and evaluated for classification accuracy across low, medium, and high susceptibility classes. Table VI presents the four models comprising the ensemble.

Table VI: ML Model Inventory

ML Model	Role and Configuration
phishing_rf (Random Forest)	100 estimators, predicts phishing susceptibility (low/med/high)
phishing_lr (Logistic Regression)	max_iter=1000, ensemble partner for phishing prediction

phishing_dt (Decision Tree)	Auxiliary interpretable model for phishing classification
account_rf (Random Forest)	100 estimators, predicts account takeover probability (0-1)

F. Testing Results

UPSA includes a pytest-based test suite across three modules: test_auth.py (authentication flows), test_risk_engine.py (risk scoring), and test_fixes.py (integration). All test cases passed. Table VII presents the testing environment configuration.

Table VII: Testing Environment Configuration

Test Configuration	Value
Test Database	SQLite in-memory (sqlite:///memory:)
CSRF Validation	Disabled (WTF_CSRF_ENABLED=False)
Rate Limiting	Disabled (RATELIMIT_ENABLED=False)
Bcrypt Rounds	4 (reduced from 12 for speed)
Mail	Suppressed (MAIL_SUPPRESS_SEND=True)
Test Runner	pytest with pytest-flask and pytest-cov

G. Technology Stack

Table VIII presents the complete technology stack. All dependencies are open-source with active maintenance. The SQLAlchemy abstraction enables migration from SQLite to PostgreSQL as a configuration change, not a code change.

Table VIII: Complete Technology Stack

Layer	Technology	Version / Notes
Backend Framework	Flask	3.x – Application Factory pattern
Language	Python	3.12+ – Type hints, modern stdlib
ORM & Database	SQLAlchemy + SQLite	Flask-SQLAlchemy, Flask-Migrate (Alembic)
Authentication	Flask-Login + Werkzeug	bcrypt, PBKDF2-SHA256 hashing
Forms & CSRF	Flask-WTF	CSRF token validation on all forms
Email	Flask-Mail	SMTP, TLS support, suppress in tests
Rate Limiting	Flask-Limiter	IP-based, fixed-window strategy
Machine Learning	scikit-learn	RF, LR, DT classifiers; joblib persist
Data Processing	pandas + numpy	ML feature engineering
Document Parsing	PyPDF2 + python-docx	PDF and DOCX text extraction
Frontend Framework	Bootstrap	5.3 – Responsive, dark/light theme
Data Visualization	Chart.js	AJAX-loaded trend charts
Testing	pytest + pytest-flask	pytest-cov for coverage reporting
Dependency Mgmt.	pip + requirements.txt	Virtual environment recommended

VI. INDUSTRY APPLICATIONS

UPSA's modular architecture positions it as a deployable security microservice or embeddable library within larger enterprise systems. Five primary integration scenarios are documented:

A. Adaptive Zero-Trust Authentication

The Risk Engine can serve as an authentication middleware layer for enterprise portals and financial platforms, dynamically scoring each login attempt and routing to the appropriate challenge level. This reduces MFA friction for recognized users while maintaining enhanced protection for anomalous access.

B. User Experience Preservation

The Frustration Analyzer provides a programmatic signal for when security controls create counterproductive friction. Enterprise systems can subscribe via the API endpoint to dynamically extend session timeouts, reduce MFA frequency, or trigger proactive outreach to high-frustration users.

C. Permission Auditing

The WebsitePermission model and contextual appropriateness evaluation provide a principled framework for identifying over-privileged browser permission grants in SaaS platforms, surfacing inappropriate permission requests to users at the point of access.

D. Security Training

The Security Awareness Survey can be embedded in corporate learning management systems. The adaptive feedback mechanism generates personalized guidance cards based on low-scoring dimensions, creating a closed loop between training completion and ML risk prediction accuracy.

E. Compliance Summarization

The Policy Analyzer can be integrated into procurement workflows, automatically generating structured risk summaries (overall_risk, data_collection_matches, user_rights_assessment) for vendor privacy policies, enabling policy risk to be incorporated into evaluation scorecards.

VII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

Location Detection: The current implementation uses a placeholder ("Unknown") for location extraction, limiting new_location and impossible_travel factor evaluation. Production deployment requires GeoIP integration (e.g., MaxMind GeoLite2).

ML Training Data: Models are trained on 500 synthetic samples. Production accuracy requires retraining on real user behavioural data with class-balanced sampling. SQLite is unsuitable for concurrent production deployments beyond a few users, necessitating PostgreSQL migration.

Policy Analyzer NLP: The rule-based keyword matching approach provides deterministic, auditable results but lacks the semantic understanding of transformer-based NLP models, potentially producing false positives on ambiguous policy language.

B. Future Enhancements

Recommended future enhancements include: (1) GeoIP integration for accurate location-based risk factor evaluation; (2) real-data ML training with privacy-preserving techniques; (3) TOTP/FIDO2 MFA replacing email-based codes for phishing-resistant second factors; (4) PostgreSQL migration for production scalability; (5) transformer-based NLP for policy analysis; (6) Redis-backed rate limiting for distributed deployments; and (7) OAuth2 / Social Login integration with appropriate risk handling for federated identity providers.

VIII. CONCLUSION

This paper presented the Unified Privacy & Security Assistant (UPSA), a comprehensive, production-ready web application that demonstrates adaptive, explainable security can achieve superior outcomes on both the usability and protection dimensions simultaneously. By treating user frustration as a first-class security signal rather than an irrelevant side effect, UPSA addresses a root cause of security policy circumvention that is widely acknowledged but rarely implemented in production systems.

Three core innovations distinguish UPSA from conventional security platforms. First, the quantification of user frustration and programmatic adaptation of security controls in response. Second, the unified Privacy Health Score providing a single, human-understandable metric aggregating eight independent security dimensions. Third, the integration of plain-language ML explanations ensuring that risk predictions are actionable rather than opaque.

Future work should prioritize GeoIP integration, retraining ML models on real behavioural data, PostgreSQL migration for production scalability, and FIDO2/WebAuthn support for phishing-resistant authentication. With these enhancements, UPSA establishes a replicable blueprint for building security systems that users will adopt and maintain — the only kind of security system that provides real-world protection.

Acknowledgment

The authors recognize the contribution of all team members. The authors also acknowledge the support of their institution in providing resources and guidance for conducting this research.

. REFERENCES

- [1] Whitten, A., & Tygar, J. D. (1999). Why Johnny can't encrypt: A usability evaluation of PGP 5.0. Proceedings of the 8th USENIX Security Symposium.
- [2] Cranor, L. F., & Garfinkel, S. (Eds.). (2005). Security and Usability: Designing Secure Systems that People Can Use. O'Reilly Media.
- [3] Brooke, J. (1996). SUS: A quick and dirty usability scale. Usability Evaluation in Industry, 189(194), 4-7.

- [4] Sauro, J. (2011). A Practical Guide to the System Usability Scale: Background, Benchmarks, and Best Practices. CreateSpace Independent Publishing.
- [5] Bonneau, J., Herley, C., van Oorschot, P. C., & Stajano, F. (2012). The quest to replace passwords. 2012 IEEE Symposium on Security and Privacy.
- [6] Adams, A., & Sasse, M. A. (1999). Users are not the enemy. Communications of the ACM, 42(12), 40-46.
- [7] Florêncio, D., & Herley, C. (2007). A large-scale study of web password habits. Proceedings of the 16th International Conference on World Wide Web.
- [8] Schaub, F., Balebako, R., Durity, A. L., & Cranor, L. F. (2015). A design space for effective privacy notices. Proceedings of the 11th Symposium on Usable Privacy and Security (SOUPS).
- [9] Palomar, E., & Tapiador, J. M. E. (2012). A survey of usable security. IEEE Security & Privacy, 10(4), 36-44.
- [10] Palun, E., & Tambi, R. (2021). Adaptive multi-factor authentication in enterprise environments. International Journal of Information Security, 20, 1-18.
- [11] McDonald, A. M., & Cranor, L. F. (2008). The cost of reading privacy policies. I/S: A Journal of Law and Policy for the Information Society, 4(3), 543-568.
- [12] Flask Documentation. (2024). Flask — A Python Microframework. <https://flask.palletsprojects.com/>
- [13] scikit-learn Developers. (2024). scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>
- [14] OWASP Foundation. (2021). OWASP Top Ten Project. <https://owasp.org/www-project-top-ten/>
- [15] NIST. (2017). Digital Identity Guidelines (SP 800-63B). National Institute of Standards and Technology.

Appendix: Demo Credentials

Table IX: Demo Credentials

Email	Password	Role
admin@upsa.local	AdminPass123!	Administrator
alice@example.com	DemoPass123!	Standard User
bob@example.com	DemoPass123!	Standard User